



Observações Gerais

- É vedado o uso de ferramentas de inteligência artificial ou modelos de linguagem (LLMs).
- A constatação de utilização de tais recursos implicará na atribuição de nota **zero**.
- O discente poderá ser chamado para apresentar e justificar o código desenvolvido.
- A reprodução parcial ou integral do trabalho de outro colega resultará igualmente em nota **zero**.
- Durante a aula, algumas resoluções dos alunos serão selecionadas por sorteio para apresentação e debate.

Retrocedendo

Retirado de: **LAMBERT, Kenneth Alfred. Fundamentals of Python: data structures. Cengage Learning, 2014.**

O **algoritmo de backtracking (retrocesso)** começa em um estado inicial predefinido e então se move de um estado para outro em pesquisa de um estado final desejado. Em qualquer ponto ao longo do caminho, quando há uma escolha entre vários estados alternativos, o algoritmo seleciona um, possivelmente de maneira aleatória e continua. Se o algoritmo alcançar um estado que representa um resultado indesejável, ele retrocede ao último ponto em que havia uma alternativa inexplorada e a experimenta. Dessa forma, o algoritmo pesquisa exaustivamente todos os estados, ou alcança o estado final desejado.

Por exemplo, suponha que você esteja caminhando pela floresta e surja uma bifurcação. Você segue o caminho à direita, que por fim chega a um beco sem saída. Nesse ponto, para continuar, você deve refazer seus passos até a bifurcação e seguir o caminho não percorrido à esquerda. Um modelo de computador dessa situação registra todas as ramificações e permite que você volte instantaneamente à bifurcação para tentar outro caminho. Existem duas técnicas principais para implementar algoritmos de retrocesso: uma usa pilhas e a outra usa recursão. O uso de pilhas é explorado a seguir.

O papel de uma pilha no processo é lembrar os estados alternativos que ocorrem em cada junção. Para ser mais preciso, o papel é o seguinte:

```
Crie uma nova pilha
Insira o estado inicial na pilha
Enquanto a pilha não estiver vazia
    Abra a pilha e examine o estado
    Se o estado representar um estado final
        Retorne CONCLUSÃO BEM-SUCEDIDA
Caso contrário, se o estado não foi visitado anteriormente
    Marque o estado como visitado
    Insira na pilha todos os estados adjacentes não visitados
Retorne CONCLUSÃO MALSUCEDIDA
```

Esse algoritmo geral de retrocesso encontra aplicações em muitos programas de jogos e resolução de quebra-cabeças. Considere, por exemplo, o problema de encontrar um caminho para sair de um labirinto. Em uma instância desse problema, um caminhante deve encontrar uma rota até o topo de uma montanha. Suponha que o caminhante saia de um estacionamento, marcado com P e explore o labirinto até chegar ao topo de uma montanha, marcado com T. A Figura 1 mostra a aparência de um possível labirinto.

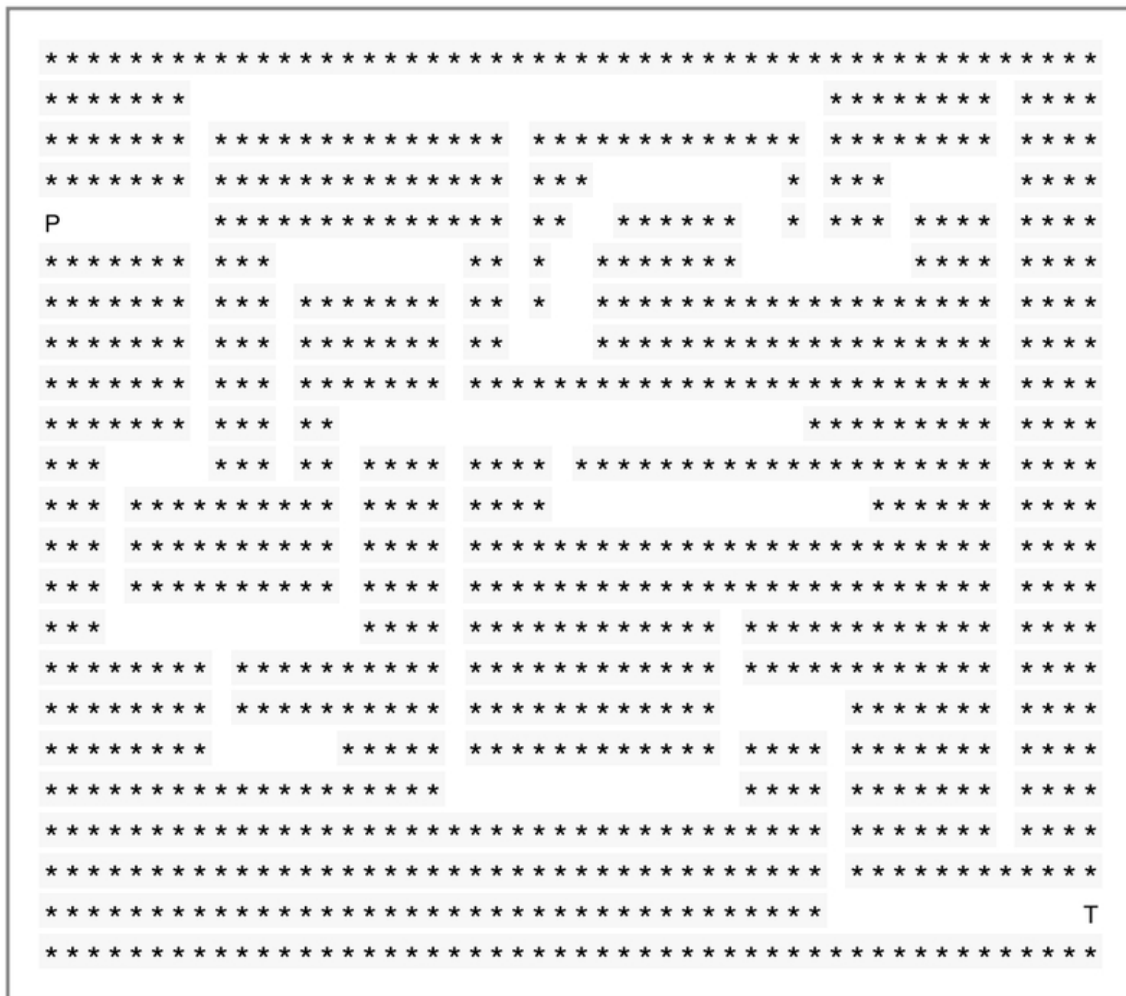


Figura 1: Um problema de labirinto.

Eis um programa para resolver esse problema. Na inicialização, o modelo de dados do programa insere o labirinto como uma grade de caracteres de um arquivo de texto. O caractere '*' marca uma barreira, e 'P' e 'T' marcam o estacionamento e o topo da montanha, respectivamente. Um espaço em branco marca um passo ao longo de um caminho. Depois que o labirinto é carregado do arquivo, o programa deve exibi-lo na janela do terminal. O programa deve então solicitar que o usuário pressione a tecla **Enter** ou **Return** para resolver o labirinto. O modelo tenta encontrar um caminho através do labirinto e retorna **True** ou **False** para a visualização, dependendo do resultado. No modelo, o labirinto é representado como uma grade de caracteres ('P', 'T', '*' ou espaço). Durante a pesquisa, cada célula visitada é marcada com um ponto. No final do programa, a grade é exibida novamente com os pontos incluídos. Eis o algoritmo de retrocesso que está no centro da solução:

```

Crie uma nova pilha
Localize o caractere 'P' na grade
Insira sua localização na pilha
Enquanto a pilha não estiver vazia
    Retire uma localização (linha, coluna) da pilha
    Se a grade contiver 'T' neste local, então
        Um caminho foi encontrado
        Retorne True
    Caso contrário, se este local não contiver um ponto
        Coloque um ponto na grade neste local
        Examine as células adjacentes a esta e para cada uma
        que contém um espaço, insira sua localização na pilha
    Retorne False

```

A partir do algoritmo apresentado, nesta atividade você deverá implementar uma solução para o problema do labirinto utilizando uma ****pilha**** para controlar o processo de exploração e retrocesso.

O labirinto será representado por uma matriz de 'String', na qual cada posição poderá conter um dos seguintes símbolos:

- "*": representa uma parede, que não pode ser atravessada;
- " ": representa uma posição livre do labirinto;
- "P": representa a posição inicial;
- "T": representa a posição final;
- "+": representa uma posição pertencente ao caminho encontrado.

O objetivo do programa é encontrar um caminho entre a posição P e a posição T. Para isso, o programa deverá explorar as posições livres do labirinto e utilizar a pilha para armazenar as posições que ainda podem ser exploradas.

Durante a exploração, quando uma posição possuir mais de uma possibilidade de movimento, uma das alternativas deverá ser explorada. Caso essa escolha leve a um caminho sem saída, o algoritmo deverá realizar o **retrocesso**, retirando posições da pilha até retornar a uma posição que possua outra alternativa ainda não explorada.

Regra fundamental

O objetivo da atividade não é simplesmente encontrar uma posição livre no labirinto. O programa deve utilizar a **pilha para controlar o avanço e o retrocesso** durante a busca pelo caminho entre P e T.

Estrutura fornecida

A matriz do labirinto será fornecida no código inicial da atividade (**disponível no sigaa**). Você deverá utilizar essa matriz como entrada do algoritmo, sem alterar a disposição das paredes, da posição inicial ou da posição final.

A classe possui, inicialmente, uma estrutura semelhante à apresentada abaixo:

```
public class Labirinto {  
  
    private char[][] mapa = {  
        // matriz fornecida pelo professor  
    };  
  
    private int linhaInicial;  
    private int colunaInicial;  
  
    private int linhaFinal;  
    private int colunaFinal;  
  
}
```

A atividade deverá utilizar a implementação de **Pilha desenvolvida nas aulas**. Não é permitido utilizar as classes prontas de Java para implementação da pilha.

Regras para movimentação

A partir de uma posição, o caminhante poderá se movimentar somente nas quatro direções:

- para cima;
- para baixo;
- para a esquerda;
- para a direita.

Não é permitido realizar movimentos diagonais.
Uma posição poderá ser explorada somente se:

- estiver dentro dos limites da matriz;
- não for uma parede ("*");
- ainda não tiver sido visitada durante a busca.

Exibição do resultado

Inicialmente, o programa deverá exibir o labirinto original.

Após a execução do algoritmo, deverá ser possível visualizar o caminho encontrado. As posições pertencentes ao caminho entre P e T deverão ser representadas pelo caractere "+" ou qualquer outra a sua escolha.

Caso não exista um caminho entre P e T, o programa deverá informar:

Nenhum caminho encontrado.

Caso exista um caminho, deverá informar:

Caminho encontrado!

Desafio

Durante a execução, observe o comportamento da pilha.

Explique, com suas palavras, o que acontece com a pilha quando o algoritmo:

1. encontra uma nova posição que pode ser explorada;
2. chega a um caminho sem saída;
3. precisa retornar a uma bifurcação;
4. encontra o destino.

Por fim, considerando um labirinto com L linhas e C colunas, determine a complexidade de tempo do algoritmo no pior caso e justifique sua resposta.